Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kỹ thuật Máy tính



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập lớn A1 - Mốc A1.1: Dataset và protocol thí nghiệm

CNN hay Transformer? So sánh ResNet-18 và DeiT-Tiny trên CIFAR-100 với nhiều chế độ huấn luyện

---

Giảng viên: Thầy Lê Thành Sách
Thầy Nguyễn Quốc Minh

Thành viên nhóm: Đàm Quang Phục - 2570483
Hồ Bảo An - 2570164

Mã nguồn:
`https://github.com/dqpinfo/Deep_Learning_Computer_Vision_CO5085/tree/main/Assignment_1`

Thành Phố Hồ Chí Minh, 10/2026

Mục lục

1 Chủ đề và mục tiêu **2**

2 Tập dữ liệu: CIFAR-100 **2**
2.1 Lý do chọn 2
2.2 Nguồn và license 2
2.3 Cách chia split 3
2.4 Khám phá dữ liệu (EDA) 3

3 DataLoader và augmentation **5**

4 Mô hình sử dụng **6**

5 Protocol thí nghiệm **7**
5.1 Ma trận 8 thí nghiệm 7
5.2 Huấn luyện 8
5.3 Metric và phân tích sẽ báo cáo ở A1.2 8

6 Kiểm tra pipeline ở mốc A1.1 **8**

7 Kế hoạch đến mốc A1.2 **9**

8 Sử dụng công cụ AI **9**

Tóm tắt mốc A1.1 (hạn 26/10/2026). Nhóm đã chốt tập dữ liệu **CIFAR-100**, cách chia train/validation/test cố định (45.000 / 5.000 / 10.000 ảnh, seed 42), DataLoader và augmentation dùng chung, hai mô hình **ResNet-18** (CNN) và **DeiT-Tiny** (Transformer), cùng ma trận **8 thí nghiệm** (2 kiến trúc × 4 chế độ huấn luyện). Pipeline đã được kiểm tra: cả 8 cấu hình tạo được, phần đóng băng đúng, vòng huấn luyện – đánh giá chạy thông trên tập con nhỏ. Kết quả số liệu cuối thuộc mốc A1.2 (09/11/2026). Mọi con số trong báo cáo lấy từ `config.py`, `output/eda/summary.json` và `output/protocol_table.json`.

1. Chủ đề và mục tiêu

Bài A1 so sánh hai họ kiến trúc cho bài toán phân loại ảnh: mạng tích chập (CNN) và Vision Transformer (ViT) [3]. Hai họ được đánh giá dưới nhiều chế độ huấn luyện: học từ đầu (*from-scratch*) so với dùng trọng số *pretrained*, và ba mức đóng băng backbone trên bản pretrained (chỉ train head, đóng băng một phần, fine-tune toàn bộ).

Mốc A1.1 chốt tập dữ liệu, cách chia split, DataLoader và augmentation, các mô hình sử dụng và kế hoạch thí nghiệm (protocol). Kết quả số liệu cuối sẽ có ở mốc A1.2 (09/11/2026).

2. Tập dữ liệu: CIFAR-100

2.1. Lý do chọn

- **Đúng ràng buộc của đề:** tập ảnh công khai, có 100 lớp (yêu cầu ≥ 10) và 50.000 ảnh huấn luyện; không trùng với các tập đã dùng ở E1 (MNIST / Fashion-MNIST / CIFAR-10).
- **Đủ khó để so sánh có ý nghĩa:** mỗi lớp chỉ có 500 ảnh train, ảnh nhỏ 32×32 và nhiều lớp rất giống nhau (ví dụ 5 loại cây: maple, oak, palm, pine, willow). Trong điều kiện này, ưu thế của pretrained và nhu cầu dữ liệu của Transformer bộc lộ rõ.
- **Vừa sức tính toán:** toàn bộ tập khoảng 140 MB; cả 8 cấu hình có thể chạy trên một GPU phổ thông (Google Colab T4) trong thời gian của mốc A1.2.
- **Có nhãn hai cấp:** 100 lớp chi tiết được gom thành 20 siêu lớp, giúp phân tích lỗi kiểu "nhầm trong cùng nhóm" hay "nhầm khác nhóm".

2.2. Nguồn và license

CIFAR-100 do Krizhevsky (2009) công bố tại `https://www.cs.toronto.edu/~kriz/cifar.html`, là tập con có gán nhãn của bộ "80 Million Tiny Images". Tập không đi kèm license chuẩn (kiểu MIT hay CC); tác giả cho phép dùng tự do cho nghiên cứu và học tập, với yêu cầu trích dẫn báo cáo kỹ thuật gốc [1]. Bộ 80 Million Tiny Images gốc đã bị gỡ năm 2020 do chứa nhãn không phù hợp, nhưng CIFAR-10/100 là tập đã được gán nhãn lại thủ công và vẫn được cung cấp chính thức.

Do server gốc tải rất chậm từ Việt Nam, nhóm tải dữ liệu từ bản sao chính thức trên Hugging Face (`uoft-cs/cifar100`). Bản này có cùng ảnh, cùng nhãn và cùng thứ tự với bản gốc. Ảnh được giải mã một lần và cache vào `data/cifar100.npz` (`data.py`); thư mục `data/` không được commit lên GitHub. Metadata của bản Hugging Face ghi sai tên lớp `crab` thành `cra`; `data.py` sửa lại tên gốc khi nạp.

2.3. Cách chia split

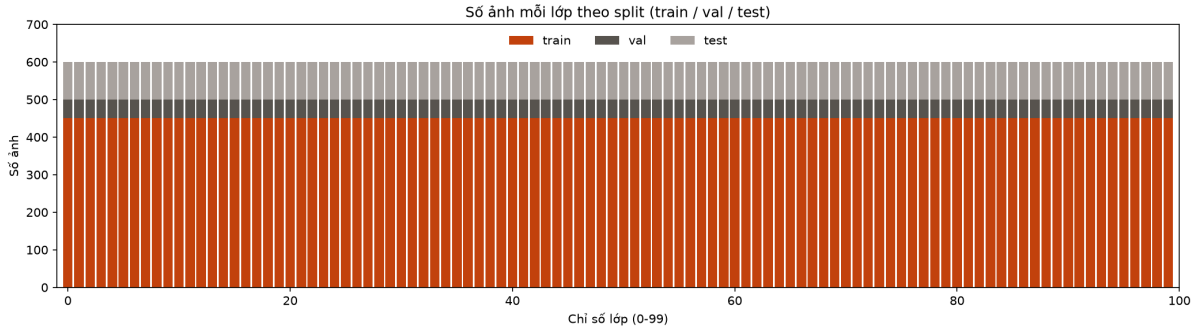
Bảng 1: Ba tập dữ liệu dùng chung cho cả 8 thí nghiệm (`output/eda/summary.json`).

Split	Số ảnh	Mỗi lớp	Nguồn	Vai trò
Train	45.000	450	Tập train gốc	Cập nhật trọng số
Validation	5.000	50	Tập train gốc	Chọn checkpoint tốt nhất, theo dõi quá trình học
Test	10.000	100	Tập test gốc	Chỉ dùng một lần ở cuối mỗi thí nghiệm

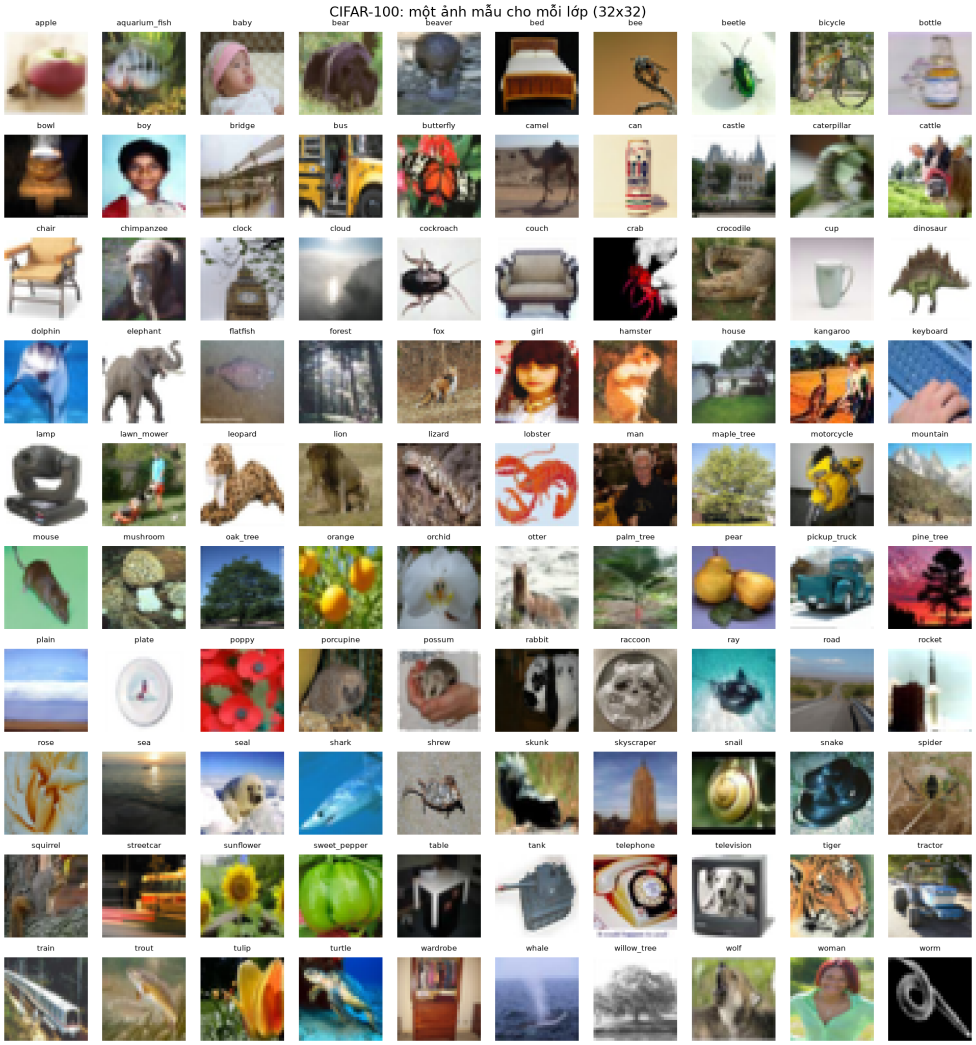
Validation được lấy **phân tầng**: mỗi lớp chọn ngẫu nhiên 50 ảnh từ tập train gốc với seed 42 (`make_split` trong `data.py`). Chỉ số ảnh được lưu cố định trong `splits/cifar100_split.json`, nên cả 8 thí nghiệm dùng **đúng cùng một split**. Tập test không được dùng để chọn siêu tham số hay checkpoint.

2.4. Khám phá dữ liệu (EDA)

Script `eda.py` đếm số ảnh mỗi lớp theo split, vẽ ảnh mẫu, ví dụ augmentation, tính thống kê pixel và liệt kê 20 siêu lớp; kết quả lưu ở `output/eda/`.



Hình 1: Số ảnh mỗi lớp theo split: dữ liệu cân bằng tuyệt đối (450 / 50 / 100 ảnh mỗi lớp).



Hình 2: Một ảnh mẫu cho mỗi lớp của CIFAR-100 (độ phân giải gốc 32×32).

Nhận xét.

- Mọi ảnh là ảnh màu RGB kích thước $32 \times 32 \times 3$. Không lớp nào thiếu ảnh (Hình 1), nên accuracy và macro-F1 phản ánh tương đương nhau; nhóm vẫn báo cáo cả hai.
- Trung bình pixel trên tập train (R, G, B) là (0.507, 0.487, 0.441), độ lệch chuẩn là (0.267, 0.256, 0.276). Thống kê này gần với thống kê ImageNet (mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)), nên việc dùng chuẩn hoá ImageNet cho mọi cấu hình là hợp lý (Mục 3).
- Ảnh rất nhỏ, vật thể thường chiếm gần hết khung hình và nhiều chi tiết bị mất. Lỗi dự kiến tập trung ở các lớp cùng siêu lớp (Bảng 2), ví dụ các loài cây, các loại xe hay các loài thú có vú cỡ nhỏ.

Bảng 2: 20 siêu lớp của CIFAR-100, mỗi siêu lớp gồm 5 lớp chi tiết.

Siêu lớp	Các lớp chi tiết
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchid, poppy, rose, sunflower, tulip

Siêu lớp	Các lớp chi tiết
food containers	bottle, bowl, can, cup, plate
fruit and vegetables	apple, mushroom, orange, pear, sweet pepper
household electrical devices	clock, keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple tree, oak tree, palm tree, pine tree, willow tree
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn mower, rocket, streetcar, tank, tractor

3. DataLoader và augmentation

Bảng 3: Pipeline tiền xử lý, giống hệt nhau cho cả 8 cấu hình (`data.py`).

Bước	Train	Validation / Test
1. Augmentation ở 32×32	`RandomCrop(32, padding=4)` + `RandomHorizontalFlip`	Không
2. Phóng ảnh	`Resize` lên 224×224 (nội suy bicubic)	
3. Chuẩn hoá	`ToTensor` + `Normalize` theo mean/std của ImageNet	
DataLoader	batch 128, `shuffle`, `drop_last`	batch 128, không shuffle



Hình 3: Một ảnh gốc và các phiên bản sau augmentation (đã phóng lên 224×224).

Lý do thiết kế.

- **Phóng lên** 224×224**:** trọng số pretrained của cả hai mô hình được học ở 224×224. Riêng DeiT-Tiny dùng patch 16×16; nếu giữ ảnh 32×32 thì chỉ còn 4 token, không đủ để attention hoạt động có ý nghĩa.
- **Cùng kích thước, cùng augmentation, cùng chuẩn hoá cho cả 8 cấu hình** (kể cả from-scratch): khác biệt kết quả chỉ đến từ ba yếu tố cần so sánh là kiến trúc, cách khởi tạo và mức đóng băng.
- **Augmentation nhẹ:** crop + flip là chuẩn phổ biến cho CIFAR. Nhóm cố ý không dùng augmentation mạnh (RandAugment, Mixup, ...) để không ưu ái riêng Transformer from-scratch, vốn hưởng lợi nhiều từ augmentation mạnh. Đây là hạn chế có chủ đích và sẽ được thảo luận ở A1.2.

4. Mô hình sử dụng

Bảng 4: Hai mô hình đại diện cho hai họ kiến trúc (`config.py`, `models.py`).

Họ	Mô hình (tên trong `timm`)	Tham số (head 100 lớp)	Lý do chọn
CNN	ResNet-18 [2] `resnet18.a1_in1k`	11.227.812	CNN chuẩn, nhỏ, có kết nối tắt (residual); baseline phổ biến nhất
Transformer	DeiT-Tiny/16 [4] `deit_tiny_patch16_224.fb_in1k`	5.543.716	Một trong các ViT nhỏ nhất có trọng số ImageNet-1k tốt, cùng cỡ với ResNet-18, chạy được trên GPU phổ thông

Hai mô hình có cùng nguồn pretrained (chỉ ImageNet-1k, không dùng ImageNet-21k), lấy qua thư viện `timm` [5], và cùng bậc kích thước, để so sánh không bị lệch vì một bên được học trước trên nhiều dữ liệu hơn. Lớp phân loại cuối của cả hai được thay bằng lớp mới 100 đầu ra, khởi tạo ngẫu nhiên.

5. Protocol thí nghiệm

5.1. Ma trận 8 thí nghiệm

Bảng 5: 8 cấu hình và số tham số trainable, đo bằng `smoke_test.py` (`output/protocol_table.json`).

Cấu hình	Kiến trúc	Khởi tạo	Chế độ	Trainable	Tỉ lệ
`resnet18_scratch`	ResNet-18	Ngẫu nhiên	Học từ đầu	11.227.812	100,0%
`resnet18_head`	ResNet-18	ImageNet-1k	Chỉ train head	51.300	0,5%
`resnet18_partial`	ResNet-18	ImageNet-1k	Đóng băng một phần	10.544.740	93,9%
`resnet18_full`	ResNet-18	ImageNet-1k	Fine-tune toàn bộ	11.227.812	100,0%
`deit_tiny_scratch`	DeiT-Tiny	Ngẫu nhiên	Học từ đầu	5.543.716	100,0%
`deit_tiny_head`	DeiT-Tiny	ImageNet-1k	Chỉ train head	19.300	0,4%
`deit_tiny_partial`	DeiT-Tiny	ImageNet-1k	Đóng băng một phần	2.688.868	48,5%
`deit_tiny_full`	DeiT-Tiny	ImageNet-1k	Fine-tune toàn bộ	5.543.716	100,0%

Định nghĩa các mức đóng băng (áp dụng trên bản pretrained, `models.py`):

- **Chỉ train head** (linear probe): đóng băng toàn bộ backbone, chỉ học lớp phân loại cuối.
- **Đóng băng một phần:** khoảng nửa đầu của mạng. ResNet-18 đóng băng `conv1`, `bn1`, `layer1`, `layer2` và train `layer3`, `layer4`, `fc`. DeiT-Tiny đóng băng `patch_embed`, positional embedding, token CLS và 6/12 block đầu; train 6 block cuối, LayerNorm cuối và head.
- **Fine-tune toàn bộ:** mọi tham số đều được cập nhật.

Hai mô hình được đóng băng cùng "một nửa độ sâu" (2/4 stage của ResNet-18, 6/12 block của DeiT-Tiny), nhưng tỉ lệ tham số trainable khác nhau: 93,9% so với 48,5%. Lý do là ResNet tập trung phần lớn tham số ở các stage sâu (`layer3`, `layer4`), còn các block của DeiT có kích thước bằng nhau. Nhóm sẽ báo cáo cả hai góc nhìn (độ sâu và số tham số) khi thảo luận ở A1.2. Với ResNet-18, các lớp BatchNorm thuộc phần bị đóng băng được giữ ở chế độ `eval` trong lúc train (`set_train_mode`), để thống kê pretrained không bị thay đổi.

5.2. Huấn luyện

Bảng 6: Siêu tham số dự kiến cho A1.2 (`config.TRAIN_HPARAMS`).

Thiết lập	From-scratch	Pretrained (3 chế độ)
Optimizer	AdamW, weight decay 0.05	
Learning rate	10^{-3}	10^{-4}
Lịch learning rate	Warmup tuyến tính 1 epoch, sau đó cosine giảm về 0	
Số epoch	30	10
Loss	Cross-entropy, label smoothing 0.1	
Batch size / seed	128 / 42	
Chọn checkpoint	Epoch có validation accuracy cao nhất; sau đó đánh giá trên test đúng một lần	

Vòng lặp huấn luyện được tự viết bằng PyTorch trong `train.py`: xoá gradient → forward → tính loss → backward → `optimizer.step()`. Thiết bị được chọn tự động theo thứ tự CUDA, MPS, CPU. From-scratch được học nhiều epoch hơn và với learning rate lớn hơn, vì mô hình phải học mọi thứ từ đầu. Để so sánh công bằng trong từng nhóm, các cấu hình cùng nhóm dùng chung siêu tham số.

5.3. Metric và phân tích sẽ báo cáo ở A1.2

- **Chất lượng:** top-1 accuracy và macro-F1 (`sklearn.metrics.f1_score`) trên validation và test; đường cong loss/accuracy theo epoch (`train.py` đã ghi đủ các giá trị này vào log JSON của mỗi lần chạy).
- **Chi phí:** thời gian huấn luyện, số tham số trainable, bộ nhớ GPU đỉnh (`torch.cuda.max_memory_allocated`).
- **Phân tích lỗi:** confusion matrix ở mức 20 siêu lớp (dễ đọc hơn ma trận 100×100), các cặp lớp hay nhầm nhất và ảnh mẫu bị đoán sai của cấu hình tốt nhất mỗi họ. Dự đoán trên test được lưu sẵn (`*_test_preds.json`) để phục vụ phân tích này.

Câu hỏi cần trả lời và giả thuyết ban đầu (sẽ kiểm chứng bằng số liệu, không coi là kết luận):

1. **ViT có cần pretrain không?** Dự đoán DeiT-Tiny from-scratch kém hơn rõ rệt so với ResNet-18 from-scratch, vì Transformer thiếu "thiên kiến cục bộ" của tích chập và CIFAR-100 chỉ có 45.000 ảnh train.
2. **Khi nào đóng băng có lợi?** Dự đoán đóng băng một phần đạt gần bằng fine-tune toàn bộ với ít tham số trainable hơn. Chỉ train head có thể kém hơn, vì ảnh 32×32 phóng lên khác khá xa ảnh ImageNet.
3. **Chi phí so với chất lượng:** so sánh accuracy trên mỗi đơn vị thời gian huấn luyện giữa các cấu hình.

6. Kiểm tra pipeline ở mốc A1.1

Script `smoke_test.py` tạo đủ 8 cấu hình, đếm tham số và chạy một bước forward + backward trên một batch 8 ảnh ngẫu nhiên 224×224. Script xác nhận các tham số bị đóng băng không nhận gradient, còn các tham số trainable thì có gradient. Cả 8 cấu hình đều đạt (cột "Kiểm tra" = OK trong `output/protocol_table.md`; số tham số ở Bảng 5).

Ngoài ra, nhóm chạy thử toàn bộ vòng lặp huấn luyện – đánh giá trên tập con nhỏ (256 ảnh mỗi tập, 1 epoch, thiết bị `mps`) để chắc chắn pipeline chạy thông: `deit_tiny_partial` (khoảng 6 giây) và `resnet18_head` (khoảng 3 giây). Huấn luyện, đánh giá validation, chọn checkpoint và đánh giá test đều chạy không lỗi; log nằm ở `output/smoke/`. Accuracy của các lần chạy này chỉ ở mức đoán ngẫu nhiên (khoảng 1–3%) vì chỉ có 1 epoch trên 256 ảnh: đây là kiểm tra code, **không phải kết quả thí nghiệm**.

7. Kế hoạch đến mốc A1.2

Bảng 7: Lịch làm việc đến hạn A1.2 (09/11/2026).

Thời gian	Công việc
27/10 – 31/10	Chạy 6 cấu hình pretrained (ResNet-18 và DeiT-Tiny × 3 mức đóng băng) trên Google Colab (GPU T4)
01/11 – 05/11	Chạy 2 cấu hình from-scratch (dài nhất, 30 epoch); lưu log và checkpoint
06/11 – 09/11	Tổng hợp bảng, vẽ biểu đồ, confusion matrix, phân tích lỗi, viết thảo luận; cập nhật báo cáo và landing page

Nếu thời gian GPU không đủ, nhóm sẽ giảm số epoch from-scratch hoặc dùng tập con train có kiểm soát (giữ đủ 100 lớp), và ghi rõ trong báo cáo A1.2.

8. Sử dụng công cụ AI

Theo quy định liêm chính học thuật của môn học, nhóm ghi rõ: trợ lý AI (Claude) được dùng để hỗ trợ viết khung mã nguồn, soạn bố cục báo cáo và rà soát lỗi. Nhóm đã đọc, chạy kiểm tra, chỉnh sửa và chịu trách nhiệm về toàn bộ nội dung; mọi lựa chọn dataset, mô hình và protocol đều được nhóm hiểu và có thể giải thích.

Tài liệu

[1] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Technical report, University of Toronto, 2009.

[2] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[3] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.

[4] H. Touvron et al. Training data-efficient image transformers & distillation through attention. *ICML*, 2021.

[5] R. Wightman. PyTorch Image Models (timm). `https://github.com/huggingface/pytorch-image-models`.